

ECE 2210: Python Programming for ECE

Programming Project 3

Description

We will assume a network intrusion detection system (NIDS), such as [snort](#), is used to monitor network traffic. Your program will take as input a sample set of data that the NIDS has processed concerning the likely threat of each packet. Your program then processes the set of threat samples and determines if there is a likely breach. The output from your program reports if it detects a breach in the sample set of data or that this set does not contain a sign of an intrusion. Your program then repeats the analysis with a new set of samples. Only when the input contains an **exit** code does your program end.

The NIDS uses a rule-based system to assign a threat level number to each packet. The threat level number is always **non-negative**, and a larger number means the packet is more likely to be malicious. The maximum threat level value is 2,147,483,647 ($= 2^{31}$). One parameter that is specified when the program begins (i.e., at run time) is a threshold level for determining if the packet is threatening. If a packet has a threat level number that is less than but **NOT** equal to the threshold, then your algorithm considers the packet as unthreatening. We define this threshold value as **min_threat_level**. False alarms can occur with the NIDS. That is, the NIDS may mistakenly classify a packet with a high threat level score even though the packet is nonthreatening. A single packet with a high threat level score does not necessarily imply that the source of the packet is an intruder. Therefore, you must verify that the threat is significant by determining if the same threat level value is assigned to multiple packets. To be considered a threat, the threat level score must be found for at least a certain threshold number of times. We define this threshold value as **false_alarm_count**, and it is also specified at run time by the user. Your algorithm takes a set of threat level samples and finds the **highest** threat level that occurs in the set enough times to not be a false alarm and with a value that is large enough so that it is threatening. For each sample set, if you detect a breach, you print the threat level for the breach and how many packets are labeled with this threat level. If a breach is not detected, then print a message that the sample is free of threats. For the required format of the print statements, see the template file.

Your program begins with a **configuration step** in which the user is prompted to input two values. The values are read in one time, and are fixed for the remaining duration of the program. That is, all the sample sets are processed using the same configuration.

1. First, prompt for the threat level threshold (**min_threat_level**) to determine whether a packet is threatening. The **min_threat_level** is an **integer** and its value must be **equal to or larger** than the constant **MINTHRESH**. If the input value is invalid (Examples of invalid input are “hello”, “5.4”, “-2”, etc.), print a message (“**The minimum level is invalid.**”) and repeat the request for the threat level threshold until a valid value is entered. However, if the value is -1, then exit your program instead (by calling **sys.exit()**).
2. Next, prompt for the false alarm threshold value (**false_alarm_count**). The **false_alarm_count** must be a **positive integer**, and again prompt until a valid value or a -1 is input.

Note that **min_threat_level** and **false_alarm_count** are defined in the given template with negative initial values. They both are expected to change during the configuration step according to the user's input.

Once the configuration step is over, the program then moves to a **sample collection phase** in which the user either enters a new set of samples or an exit code.

- 1) If a value is a **non-negative integer**, it is a threat level score for the current set.
- 2) If the value is **-1** then the sample input is complete and your code should evaluate this sample set for a threat, print the result, and then begin to collect a new sample set.
- 3) If the value is **any other negative value** then the program immediately quits (by calling **sys.exit()**) and any samples that are part of an incomplete set are discarded.

ECE 2210: Python Programming for ECE Programming Project 3

Notes

1. Use of Dictionaries.
Although we haven't covered dictionaries yet, you can take the initiative to familiarize yourself with some of its basic uses. You may find it the most suitable tool to keep track of the number of occurrences of different threat values in a sample.
2. Both **MAXSAMPLES** and **MINTHRESH** are *global variables*, which can be accessed/changed by any function defined in the same Python script. In this project, these two variables are used to hold two constants that should not be changed. To tell other programmers that a given value should be treated as a constant, you should use a widely accepted naming convention for the constant's identifier or name. You should write the name in capital letters with underscores separating words, as stated in the Constants section of PEP 8.

```
MAXSAMPLES = 500
MINTHRESH = 3
```

Note **MAXSAMPLES** is the maximum number of samples that can be input for one sample set. A sample set is complete when either exactly **MAXSAMPLES** are entered, or the input value is -1.

3. Here is an example of the input a user might type (the format is critical), and the output that must be printed. The user types the numbers in the left column (one number at a time followed by the Enter key). We will test with extensive combinations of inputs that will try to break your program.

Sample Input	Minimum Required Output for the Sample Input
3.2	The minimum threat level is invalid. What is the minimum threat level? The minimum threat level is 5. The false alarm threshold is invalid. What is the false alarm threshold? A false alarm if the count is < 3.
5	
-4	
3	
10	The threat is invalid. What is the threat?
5	
4.0	
2	
5	
50	
5	
-1	Threat detected with level 5 and appears 3 times
10	The threat is invalid. What is the threat?
5	
5	
10	
0.8	
5	
10	
-1	Threat detected with level 10 and appears 3 times
4	
4	
4	

ECE 2210: Python Programming for ECE

Programming Project 3

4	
-1	No threat detected
1000	
5	
1000	
5	
5	
5	
10	
20	
-1	Threat detected with level 5 and appears 4 times
10	
-1	No threat detected
-2	Goodbye

4. Code Testing:

Multiple input (**test_input**) and output (**output**) examples are given. You can use any text editor (e.g., Sublime) to open an output file to see what the expected output looks like. To test your code, run it in the Linux terminal using the following command with a **test_input** and its corresponding **output** in the same directory, which generates a text file called **myoutput** that contains the output of your program.

```
$ python3 pp3.py < test_input > myoutput
```

Then, run the following command in Linux terminal to compare the output of your program and the given output.

```
$ diff output myoutput
```

If there is no output of running diff, it means the **output** and **myoutput** are identical. Note that TA will use the same command to test your program.

5. To receive credit for this assignment your code must run. Code that crashes will receive a **0**. Code that correctly detects the given examples will be scored at least 50% for correctness.

Submit your completed script **pp3.py** to Canvas.

Work must be completed by each individual student. See the “**programming_guide.pdf**” for additional policies.